


# Lesson 3 Teacher's Script

Programming Basics II: Take Your Shot

VIDEO GAME CREATION

## [0:00-0:05] Welcome Back

"Welcome back! Last week you built a Key Press Reactor and started your Semester 1 project. Quick show of hands - who got their character moving in all 4 directions? Anyone try one of the Level 3 stretch ideas - blinking, sound, recoloring?" [Pause, call on 1-2 volunteers to describe what they tried]

"Today we're adding the next real piece of your game: shooting. By the end of today, pressing a key will fire something off your character and up the screen. This is also going to teach us something new and important - what happens when your game needs to create new things while it's running, not just at the start."

## [0:05-0:25] Instances: One Template, Many Copies

"Here's a question: when you press your fire button five times fast, you want five separate bullets on screen, right? Not one bullet that teleports around. So where do those five bullets come from?" [Pause for guesses]

"The answer is something almost every engine calls the same thing: an instance. You build ONE definition of what a bullet is - what it looks like, how it behaves - and then, every time you press fire, the engine stamps out a brand new, completely independent COPY of that definition. That's an instance. Change one bullet's position, and it doesn't affect any of the others - they're all separate."

[Demo cue: switch to Godot, open a simple project with a Bullet scene already built]

"Watch the vocabulary here, because it's almost identical across every engine we use. In Godot, the function is literally called instantiate(). " [Show a line of code: var b = bullet_scene.instantiate()] " In GameMaker it's instance_create_layer(). In Unity it's Instantiate() - same word, capital I. Unreal calls it Spawn Actor from Class. All of them mean the exact same thing: take a template, make a new independent copy, place it in the game."

[Live demo] "Let's run this and press the fire key a few times." [Press fire repeatedly, point at multiple bullets on screen] "See how they're all moving independently? Each one remembers its own position separately from all the others, and separately from the template they were copied from."

[Discussion question] "Where else in a game might you want lots of independent copies of one thing? " (Examples to draw out if needed: enemies, coins/pickups, particles, trees in a forest.)

*[Timing checkpoint: wrap by 0:25]*

## [0:25-0:45] Cleanup: Why You Have to Remove What You Create

"Now here's the part that trips people up. If every fire button press creates a brand new bullet, and you fire a hundred times over the course of a level... what happens to all those old bullets once they've flown off the top of the screen?" [Pause for guesses]

"If you don't do anything - they just stay. Forever. Even after they're invisible, off-screen, and completely useless to the player, the engine is still tracking every single one, still checking its position every frame, still drawing it if it's on-screen. A hundred forgotten bullets doesn't sound like much, but a real game spawning things constantly - bullets, particles, enemies - can pile up thousands of forgotten objects in minutes, and the game slows to a crawl or crashes."

[Demo cue: same Godot project] "This is why every engine also gives you an explicit way to remove an instance once you're done with it. In Godot it's called queue_free(). GameMaker calls it instance_destroy(). Unity and Unreal both literally call it Destroy(). Different names, same job: tell the engine 'I'm done with this one, take it off the list.'"

[Live demo] "Watch what happens if I comment out the cleanup line and fire a bunch of times." [If feasible, show a debug counter or the scene tree growing; otherwise describe it] "See the count climbing and never coming back down? Now watch with cleanup turned back on." [Re-enable, fire again, show the count staying flat as bullets appear and disappear]

"So the pattern for anything you spawn at runtime is always two-sided: something that creates it, and something that eventually destroys it. Today, that second half is triggered by the bullet leaving the screen - but later this semester, you'll destroy things for other reasons too, like an enemy getting hit, or a pickup getting collected."

[Discussion question] "Can anyone think of a game where you've noticed things get removed after they're used or off-screen? " (Examples: arrows in a shooting game, particle bursts, collected coins.)

*[Timing checkpoint: wrap by 0:43-0:45]*

## [0:45-0:50] One Press, One Shot

"Quick, important one before we go build: when you check whether a key is pressed, there are actually two different questions you could be asking. 'Is this key currently being held down right now?' - that's true for as long as you hold it, checked fresh every single frame. Or: 'did this key just get pressed, this exact frame, for the first time?' - that's only true for one single frame, no matter how long you hold it after."

"For movement, back in Lesson 2, you wanted the 'currently held' version - hold the key, keep moving. But for firing a bullet, if you use that same 'currently held' check, holding the fire key down for one second at 60 frames a second fires sixty bullets in that one second. That's almost never what you want."

[Demo cue: same Godot project] "So firing needs the other check - 'just pressed, this one frame.' In Godot that's is_action_just_pressed(). GameMaker has keyboard_check_pressed() alongside keyboard_check(). Unity's Keyboard.current.spaceKey.wasPressedThisFrame versus .isPressed. Same split, different names, in basically every engine."

"Your handout today already uses the right one for each engine, so you won't have to guess - but now you'll understand WHY it's written that way instead of just copying it."

*[Timing checkpoint: wrap by 0:50]*

## [0:50-1:00] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one. Everything today gets added on top of the character you already built."

"The goal: pressing your fire key spawns a projectile at your character, it flies up the screen on its own, and it cleans itself up once it's gone. Level 1 is getting that working once. Level 2 is making sure it works cleanly every time you press it, with a cooldown so holding the key doesn't flood the screen. Level 3 is polish - a sound, a distinct look for the bullet, or tidying up how the code is organized."

[Hand out engine-specific handouts]

"One more thing before you start: if your character from Lesson 2 still moves in all 4 directions, that's totally fine to leave as-is. Our actual game only needs Left and Right, so it's also fine to trim it down if you'd rather match the real game exactly - your call, either way works."

"Questions before we start lab? ... Let's go build something that shoots."

## Lab Time Notes

- The single most common bug this week: a held-down fire key spawning dozens of bullets per second. If a student's screen fills up with bullets instantly, this is almost always the cause - check whether they're using the "held" input check instead of the "just pressed" one.
- Second most common: bullets that spawn but never move, or move but never disappear - in every engine, this usually means the movement/cleanup code ended up in the wrong event (e.g. a Create event instead of a Step event, or vice versa).
- Circulate by engine cluster, starting with whichever engine has the most students this week.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.
- At 1:50, call time and do a 2-minute share-out: ask 2-3 students to show what they built, across different engines if possible.